

Data Mining & Machine Learning
M.Sc. in Artificial Intelligence and Data Engineering
University of Pisa

Project Report

Academic Year 2025–2026

Project Title: PhishingSphere

Authors

Mattia Correnti — Student ID: 665367
Francesco Ladu — Student ID: 722538

Submission date: 12/09/2026

Abstract

This study addresses phishing-website detection as a binary classification task using the UCI Phishing Websites dataset, containing 11,055 observations and 30 discrete predictors. Three representations of repeated data profiles were investigated, including a weighted profile-deduplicated setting based on majority labels and retained-support sample weights.

Decision Tree and Random Forest were compared through leakage-aware nested stratified cross-validation, jointly optimizing Mutual Information feature selection and classifier hyperparameters. In the weighted experiment, Random Forest achieved a mean outer-fold accuracy of 0.9578, significantly outperforming Decision Tree (0.9359, $p = 0.0039$). The final Random Forest retained all 30 predictors and achieved a weighted test accuracy of 0.9612, macro F1-score of 0.9606, and ROC–AUC of 0.9937. Permutation importance and SHAP showed that predictions rely on multiple complementary signals, with URL_of_Anchor and SSLfinal_State among the most influential features.

1. Introduction

Phishing websites imitate legitimate services to obtain credentials or other sensitive information. Machine-learning methods can support their detection by combining patterns related to URLs, webpage content, domain configuration, and reputation [2, 3]. Reliable evaluation is particularly important because false negatives leave phishing websites undetected, while false positives incorrectly flag legitimate websites. Repeated or conflicting predictor profiles may also produce optimistic performance estimates if not handled appropriately.

This project aims to:

1. investigate the effect of different treatments of repeated feature profiles;
2. compare Decision Tree and Random Forest through nested cross-validation with joint feature selection and hyperparameter optimization;
3. evaluate and interpret the selected model through test performance, global and local explainability, and error analysis.

2. Problem

The project addresses phishing website detection as a binary classification task using the UCI Phishing Websites dataset, containing 11,055 observations and 30 discrete predictors. The target, `Result`, distinguishes phishing (−1) from legitimate (1) websites [1].

Formally,

$$f : \mathcal{X} \subseteq \{-1, 0, 1\}^{30} \rightarrow \{-1, 1\}.$$

Table 1 summarizes the dataset and evaluation setting. Three representations of repeated predictor profiles are investigated in Section 3.

Table 1. Problem and dataset at a glance.

Attribute	Summary
Task type	Binary classification
Dataset	UCI Phishing Websites — 11,055 original observations
Target variable	Result: phishing (−1) or legitimate (1)
Features	30 discrete binary and ordinal attributes from URL, HTML/JavaScript, domain, and reputation data
Missing values	None (0% globally and within both classes)
Class distribution	Phishing 44.31%; Legitimate 55.69% — approximately balanced
Evaluation metrics	Primary: accuracy; secondary: macro F1-score, phishing precision, phishing recall, and ROC–AUC
Evaluation protocol	Held-out test set; nested cross-validation on the development set

2.1 Exploratory Data Analysis

EDA was performed on the raw development set, while the held-out test set remained untouched. The development set contains 8,844 observations: 3,918 phishing (44.3%) and 4,926 legitimate (55.7%), confirming an approximately balanced target distribution.

All predictors are low-cardinality discrete variables. Figure 1 shows their class-conditional distributions, ordered by Mutual Information. The strongest associations with the target are SSLfinal_State (0.347) and URL_of_Anchor (0.333), followed by Prefix_Suffix (0.086). No missing values were found. Among the 8,844 development observations, 5,093 distinct predictor profiles are present; 57 profiles occur with both labels, involving 270 observations (3.05%). This motivated the alternative duplicate-handling strategies in Section 3.

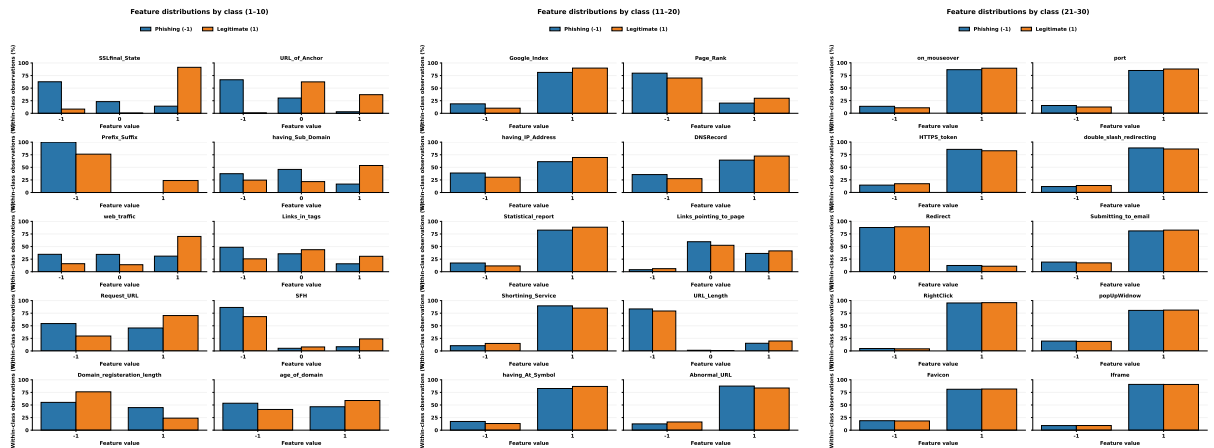
**Figure 1.** Class-conditional distributions of the 30 predictors, ordered by decreasing Mutual Information.

Figure 2 shows the 10 predictors with the strongest associations, ranked by each predictor’s maximum absolute Spearman correlation with any other predictor. The strongest positive correlations are Favicon–popUpWindow ($\rho = 0.940$), Shortening_Service–double_slash_redirecting ($\rho = 0.837$), and Favicon–port ($\rho = 0.804$), indicating redundancy among some features. Since no pair achieved the threshold of 0.95, we retained all 30 features.

2.2 Feature Description

The dataset consists of 30 discrete categorical predictors extracted from URL structures, webpage source code, domain configurations, and external reputation services [2]. Table 2

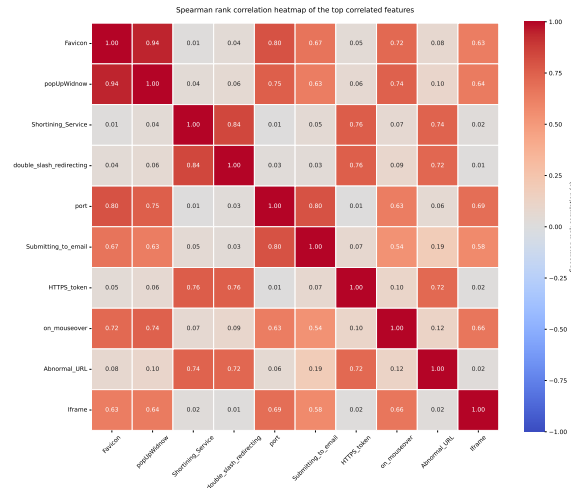


Figure 2. Spearman correlation matrix of the 10 predictors with the strongest pairwise correlations.

summarizes the four thematic categories, their associated predictors, and their predictive rationale.

Table 2. Phishing Website Prediction Feature Categories [2].

Category	Count	Features	Meaning and Predictive Rationale
Address Bar	12	having_IP_Address, URL_Length, Shortining_Service, having_At_Symbol, double_slash_redirecting, Prefix_Suffix, having_Sub_Domain, SSLfinal_State, Domain_registration_length, Favicon, port, HTTPS_token	Identifies deceptive patterns, IP literals, domain prefix/suffix hyphens, shortened URLs, and SSL certificate validity.
Abnormal URLs	6	Request_URL, URL_of_Anchor, Links_in_tags, SFH, Submitting_to_email, Abnormal_URL	Detects abnormal resource requests, off-domain anchor references, blank form actions (SFH), and email-based form submissions.
HTML / JS	5	Redirect, on_mouseover, RightClick, popUpWidnow, Iframe	Flags client-side manipulation, such as invisible iframes, disabled right-click handlers, mouseover status bar masking, and fake popup windows.
Domain / Reputation	7	age_of_domain, DNSRecord, web_traffic, Page_Rank, Google_Index, Links_pointing_to_page, Statistical_report	Measures domain longevity, valid DNS records, web traffic rank, search indexation, and presence on black-list/phishing reports.

3. Pipeline and Experiments

3.1 Preprocessing

The artificial index column was removed and Result was separated from the predictors (data_preprocessing/data_processing.py, lines 11–25, 87–131). No imputation, encod-

ing, or scaling was required because the dataset contains no missing values, discrete integer codes already represent the predictors, and both classifiers are tree-based.

Three data representations were evaluated under the same model-selection framework, differing only in the treatment of repeated predictor profiles. This choice was motivated by an inherent ambiguity in the dataset. Since the predictors are provided as already discretized attributes, two observations with identical feature values cannot necessarily be assumed to correspond to the same website. They may instead represent distinct websites that share the same discretized characteristics. Conversely, repeated profiles may also originate from genuinely duplicated observations. Therefore, removing all repeated profiles a priori could discard potentially meaningful information, while retaining them without further analysis could overrepresent frequently occurring profiles and lead to a less restrictive evaluation setting. To avoid imposing either assumption, three experimental representations were considered:

- **Experiment 1 – Raw data:** all 11,055 original observations were retained.
- **Experiment 2 – Exact deduplication:** rows sharing both predictors and target were collapsed, yielding 5,849 observations.
- **Experiment 3 – Weighted profile deduplication:** identical predictor profiles were grouped, the majority label was retained with its support as `sample_weight`, and exact ties were discarded, yielding 5,772 profiles.

The three dataset representations and their subsequent stratified development–test splits are implemented in `clean_and_split.py`: the stratified split is defined at lines 104–122, exact feature–target deduplication at lines 125–140, and weighted profile deduplication at lines 143–272. The three experiment-specific preparation procedures are implemented at lines 303–432. Each representation was stratified into development and held-out test sets: 8,844/2,211 observations for Experiment 1, 4,679/1,170 for Experiment 2, and 4,617/1,155 profiles for Experiment 3. Test sets were excluded from all model-selection steps.

3.2 Feature Selection and Engineering

Mutual Information feature selection was implemented with `SelectKBest` as the first pipeline step, followed by Decision Tree or Random Forest (`shared/modeling.py`, lines 118–147). The number of retained features was jointly optimized with classifier hyperparameters:

$$k \in \{5, 10, 15, 20, 25, \text{all}\}.$$

To prevent leakage, `SelectKBest` was fitted independently within each cross-validation training fold; validation data therefore did not affect either feature ranking or selection (`model_selection/engine.py`, lines 170–278).

Experiments 1 and 3 ultimately retained all 30 predictors, whereas Experiment 2 retained 25, excluding `popUpWindow`, `RightClick`, `on_mouseover`, `Iframe`, and `Submitting_to_email`, the five lowest-ranked features. Figure 3 shows the Mutual Information ranking of the ten predictive features on the development data. `SSLfinal_State` and `URL_of_Anchor` clearly dominate the univariate ranking. However, the final Experiment 3 pipeline retained all predictors, suggesting that features with weaker individual associations still provide complementary information to the Random Forest.

3.3 Model Selection and Cross-Validation

A decision tree was used as a tree-based baseline and Random Forest as an ensemble alternative with lower variance and the ability to model non-linear relationships and feature

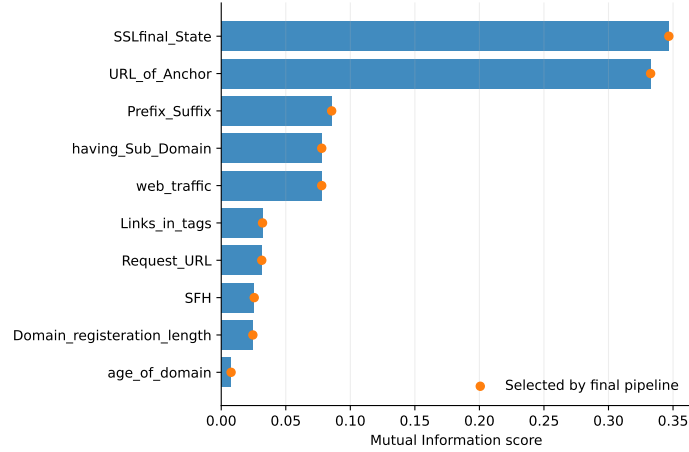


Figure 3. Top-10 predictors ranked by Mutual Information with the target on the development data.

interactions.

Model selection used nested stratified cross-validation with 10 outer folds for performance estimation and 5 inner folds for joint feature selection and hyperparameter optimization. Within each experiment, DT and RF used the same outer folds, enabling paired comparisons (`model_selection/engine.py`, lines 23–33; `model_selection/main.py`, lines 293–322).

All data-dependent operations were fitted within the training folds, preventing validation data from influencing feature selection or model tuning (`model_selection/engine.py`, lines 170–276). No resampling or class weighting was required because the classes are approximately balanced. Experiments 1–2 used unit weights during fitting and standard accuracy for scoring, whereas Experiment 3 used retained-support sample weights during fitting and retained-support weighted accuracy for scoring. Scores are therefore compared primarily within each representation. RF achieved higher outer-fold accuracy in all three experiments and was selected for the final hyperparameter search. Figure 4 shows a consistent upward shift from Decision Tree to Random Forest across all three representations. The largest separation occurs in Experiment 3, while the paired statistical tests reported in Section 3.5 confirm that the improvements are statistically significant.

Table 3 and Figure 4 compare the two model families.

Table 3. Comparison of candidate models across the three experiments.

Model	HP tuning	CV strategy	Val. metric	Test metric
DT / Exp. 1	Grid Search	Nested 10/5-fold	0.9604	0.9711
RF / Exp. 1	Random Search	Nested 10/5-fold	0.9699	0.9760
DT / Exp. 2	Grid Search	Nested 10/5-fold	0.9299	0.9368
RF / Exp. 2	Random Search	Nested 10/5-fold	0.9455	0.9487
DT / Exp. 3	Grid Search	Nested 10/5-fold	0.9359	0.9360
RF / Exp. 3	Random Search	Nested 10/5-fold	0.9578	0.9612

3.4 Hyperparameter Optimization

Feature selection and classifier hyperparameters were optimized jointly. Experiments 1–2 used accuracy as the optimization metric, whereas Experiment 3 used retained-support weighted accuracy.

Decision Tree used Grid Search over `feature_selection__k` $\in \{5, 10, 15, 20, 25, \text{all}\}$,

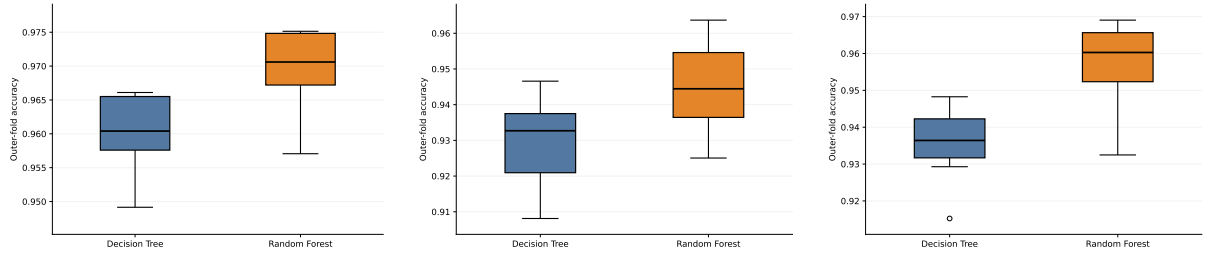


Figure 4. Outer-fold accuracy distributions for DT and RF across the three experiments. Experiment 3 uses retained-support weighted accuracy.

$\text{criterion} \in \{\text{gini}, \text{entropy}\}$, $\text{max_depth} \in \{\text{None}, 3, 5, 7, 9, 12\}$, $\text{min_samples_split} \in \{2, 4, 6, 8\}$, and $\text{min_samples_leaf} \in \{1, 2, 4\}$.

Random Forest used Randomized Search with 180 configurations over $\text{feature_selection_k} \in \{5, 10, 15, 20, 25, \text{all}\}$, $\text{n_estimators} \in \{100, 200, 400, 600, 800\}$, $\text{criterion} \in \{\text{gini}, \text{entropy}\}$, $\text{max_depth} \in \{\text{None}, 5, 10, 15, 20, 30\}$, $\text{min_samples_split} \in \{2, 4, 6, 10\}$, $\text{min_samples_leaf} \in \{1, 2, 4\}$, $\text{max_features} \in \{\text{sqrt}, \text{log2}, 0.5, \text{None}\}$, and $\text{bootstrap} \in \{\text{True}, \text{False}\}$.

The one-standard-error rule selected among competitive configurations, preferring fewer features and then shallower trees (`model_selection/utils.py`, lines 98–139). After nested CV, a final five-fold stratified search on the complete development set selected the frozen configuration used for test evaluation. Table 4 reports the final Random Forest configuration selected for each experiment. Figure 5 shows that competitive validation performance is obtained across several feature-count and depth combinations rather than at a single isolated configuration. For Experiment 3, the final procedure selected all 30 predictors with $\text{max_depth}=20$.

Table 4. Final Random Forest hyperparameters selected across the three experiments.

Hyperparameter	Exp. 1	Exp. 2	Exp. 3
<code>feature_selection_k</code>	all	25	all
<code>n_estimators</code>	800	800	600
<code>criterion</code>	entropy	entropy	gini
<code>max_depth</code>	None	10	20
<code>min_samples_split</code>	2	6	6
<code>min_samples_leaf</code>	1	1	1
<code>max_features</code>	log2	None	log2
<code>bootstrap</code>	False	True	False

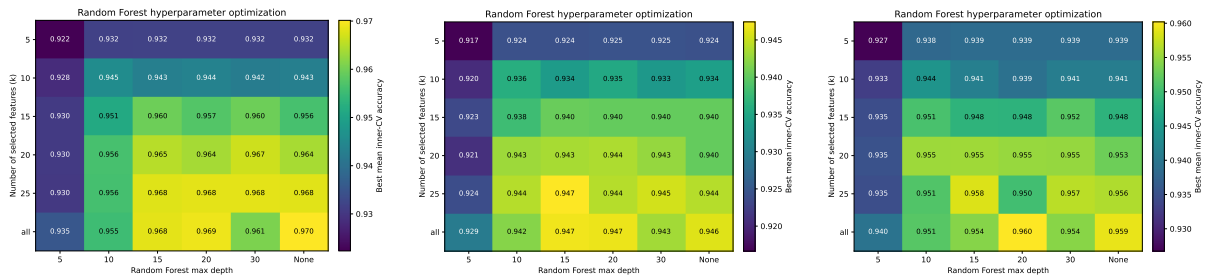


Figure 5. Final Random Forest hyperparameter search across the three experiments. Experiment 3 uses retained-support weighted accuracy.

3.5 Statistical Tests

A two-sided Wilcoxon signed-rank test compared DT and RF using paired accuracies from the same 10 outer folds (`model_selection/summaries.py`, lines 30–76). RF significantly outperformed DT in all experiments: Exp. 1 ($W = 0$, $p = 0.00195$), Exp. 2 ($W = 0$, $p = 0.00195$), and Exp. 3 ($W = 1$, $p = 0.00391$). The corresponding mean improvements were 0.0095, 0.0156, and 0.0219, respectively.

3.6 Final Model Training

After the final five-fold search, the selected pipeline was refitted on the complete development set using the frozen hyperparameters and evaluated on the corresponding held-out test set (`final_evaluation/main.py`, lines 108–211). Experiment 3 used retained-support sample weights, whereas Experiments 1–2 used unit weights (`final_evaluation/main.py`, lines 349–432). No model-selection decision was modified after observing test performance.

4. Results and Discussion

4.1 Performance Evaluation

Random Forest was selected over Decision Tree in all three experiments based on the nested cross-validation results reported in Section 3.3. On the corresponding held-out test sets, Random Forest achieved accuracies of 0.9760, 0.9487, and 0.9612 for Experiments 1, 2, and 3, respectively. The corresponding ROC–AUC values were 0.9945, 0.9899, and 0.9937.

These values should not be interpreted as a direct ranking of the three dataset representations, since each experiment produces a different development–test partition. In addition, Experiment 1 retains repeated predictor profiles and therefore represents a less restrictive evaluation setting, where the presence of duplicated observations may lead to a more optimistic estimate of predictive performance. Experiment 2 removes such repetitions, providing a more conservative evaluation, but it also discards the information associated with their original frequency.

Experiment 3 was selected for the detailed final analysis because it removes repeated predictor profiles while preserving their empirical support through sample weights. By combining deduplication with information about the original data distribution, it is expected to provide a more robust estimate of generalization performance. Its final Random Forest achieved a macro F1-score of 0.9606, phishing precision of 0.9581, and phishing recall of 0.9538. Its test accuracy of 0.9612 was also close to the nested cross-validation estimate of 0.9578, indicating a limited difference between development and held-out performance.

A comparison with previous work provides additional context for these results. Shabudin et al. [3] evaluated Random Forest on the same UCI Phishing Websites dataset using 10-fold cross-validation, feature selection, and parameter tuning. Their optimized Random Forest with Feature Selection by Omitting Redundant Features retained 22 predictors and achieved an accuracy of 0.9718. Our Experiment 1, which retains the original observations and is therefore the closest representation for comparison, achieved a held-out test accuracy of 0.9760. The two results are of a similar magnitude, but they should not be interpreted as a direct ranking because the evaluation protocols differ: Shabudin et al. report 10-fold cross-validation performance, whereas our model-selection procedure uses nested 10/5-fold cross-validation followed by evaluation on a separate held-out test set. Furthermore, our Experiments 2 and 3 explicitly

investigate the effect of repeated predictor profiles, which was a separate objective of this study.

Figure 6 shows the retained-support weighted confusion matrix for Experiment 3, while Figure 7 shows the corresponding ROC curve on the held-out test set. The model correctly classified 868 phishing and 1,113 legitimate observations, while 42 phishing observations were classified as legitimate and 38 legitimate observations were classified as phishing. False negatives are particularly relevant in this application because they allow phishing websites to remain undetected, whereas false positives may incorrectly block legitimate websites.

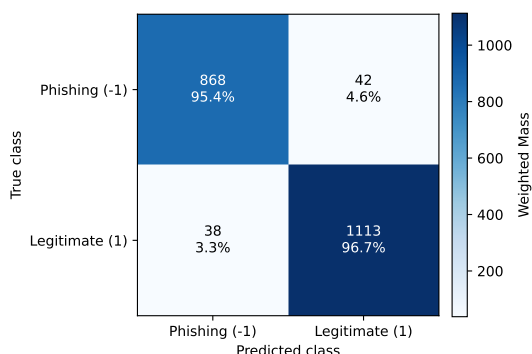


Figure 6. Retained-support weighted confusion matrix of the final Random Forest for Experiment 3 on the held-out test set.

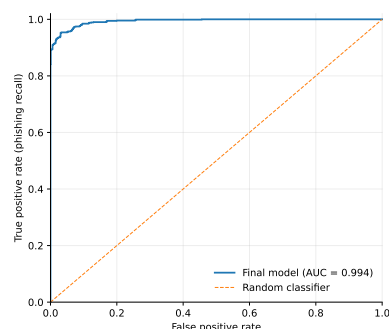


Figure 7. ROC curve of the final Random Forest for Experiment 3 on the held-out test set.

4.2 Model Explainability

Global model behavior was analyzed using permutation importance computed on the outer validation folds of the nested cross-validation procedure and then aggregated across folds. Figure 8 shows that URL_of_Anchor and SSLfinal_State have the largest effects on predictive performance, consistently with their strong individual associations observed during EDA. Their relevance is also consistent with the application domain: suspicious anchor behavior may reveal deceptive navigation, while SSL-related information provides evidence about connection legitimacy. The remaining predictors have smaller individual effects, indicating that the final classification also depends on several secondary signals.

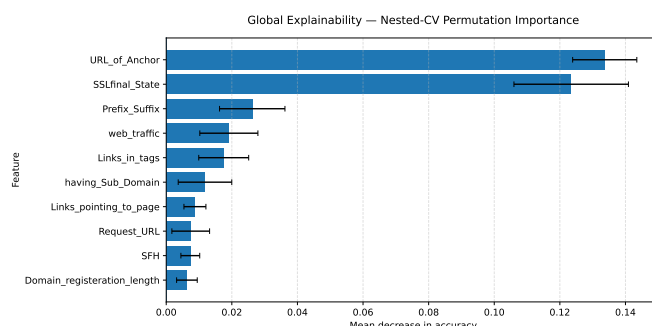


Figure 8. Global permutation importance for the Random Forest in Experiment 3, aggregated across the outer cross-validation folds.

Local model behavior was analyzed using SHAP. Figure 9 compares a correctly classified phishing profile with a false-negative profile from the held-out test set. Red contributions increase the predicted phishing probability, whereas blue contributions decrease it.

In the correctly classified case, SSLfinal_State=0 and URL_of_Anchor=-1 strongly increase

the predicted phishing probability, resulting in $P(\text{phishing}) = 0.99$. The dominant local contributions are therefore consistent with the global importance ranking.

The false-negative case shows a different combination of signals. `SSLfinal_State=1`, `URL_of_Anchor=0`, and `having_Sub_Domain=1` decrease the predicted phishing probability. Although `Links_in_tags=-1` and `web_traffic=0` provide evidence toward phishing, their contributions are insufficient to offset the signals toward legitimacy, resulting in $P(\text{phishing}) = 0.26$. This comparison shows that influential predictors can support different classes depending on their values and on the contributions of the remaining features.

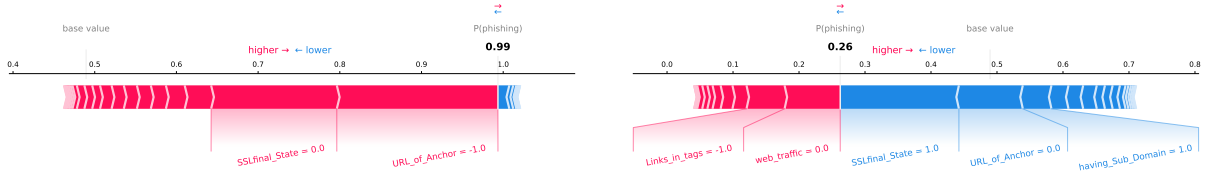


Figure 9. Local SHAP explanations for two held-out test profiles from Experiment 3. Left: a correctly classified phishing profile ($P(\text{phishing}) = 0.99$). Right: a false-negative profile ($P(\text{phishing}) = 0.26$). Red contributions increase the predicted phishing probability, whereas blue contributions decrease it.

4.3 Error Analysis

The final Random Forest misclassified 44 of the 1,155 unique test profiles in Experiment 3 (26 false negatives and 18 false positives), corresponding to a weighted error mass of 80 out of 2,061 observations. To identify recurring failure patterns, out-of-fold predictions from nested cross-validation were analyzed by feature value.

The largest weighted error rates occurred for `web_traffic=0` (8.44%) and `URL_of_Anchor=0` (8.19%). In comparison, `URL_of_Anchor=-1` and `1` produced error rates of only 0.34% and 0.95%, respectively. The intermediate anchor configuration is therefore substantially more ambiguous for the classifier than the two extreme values.

A second pattern concerns `SSLfinal_State`. Profiles with `SSLfinal_State=1` contained 96 false-negative profiles in the out-of-fold predictions, whereas profiles with `SSLfinal_State=-1` contained 47 false-positive profiles. Errors therefore often occur when the SSL-related signal points in a direction that conflicts with the true class. Similarly, `Links_in_tags=-1` showed a weighted error rate of 5.86%, indicating that some webpage-level signals remain ambiguous when combined with the other predictors.

The false-negative example in Figure 9 provides a local instance of these aggregate failure patterns. In particular, `SSLfinal_State=1` and `URL_of_Anchor=0` push the prediction toward legitimacy, while the phishing-related contributions of `web_traffic=0` and `Links_in_tags=-1` are not sufficient to reverse the prediction. This is consistent with the higher error rates observed for these feature values and shows how conflicting signals can produce a security-critical false negative.

The difference between profile-level and weighted errors also shows that some misclassified profiles occur repeatedly in the original data. Moreover, the EDA identified identical predictor profiles associated with opposite labels. Although Experiment 3 assigns each non-tied repeated profile its majority label, these conflicts show that the available 30-feature representation does not always uniquely determine the class.

Taken together, the global importance, local SHAP explanations, and feature-level error analysis indicate that the remaining errors are concentrated in profiles containing ambiguous or conflicting signals rather than being attributable to a single systematic failure of the classifier.

5. Conclusions

This project investigated phishing detection under three treatments of repeated predictor profiles. Random Forest consistently outperformed Decision Tree under nested cross-validation, with statistically significant paired differences in all experiments. Experiment 3 was selected for the detailed final analysis because it combines profile deduplication with retained-support sample weights. Its final Random Forest retained all 30 predictors and achieved a weighted test accuracy of 0.9612, macro F1-score of 0.9606, phishing recall of 0.9538, and ROC–AUC of 0.9937.

The three project objectives were addressed by comparing duplicate-handling strategies, selecting models through a common leakage-aware framework, and analyzing performance, global and local explanations, and failure modes. The main limitations are the age of the dataset, its hand-designed and external reputation features, and ambiguous predictor profiles. Future work should evaluate the pipeline on more recent, temporally separated data and richer feature representations.

References

- [1] R. Mohammad and L. McCluskey. *Phishing Websites*. UCI Machine Learning Repository, 2012. DOI: [10.24432/C51W2X](https://doi.org/10.24432/C51W2X).
- [2] R. M. Mohammad, F. Thabtah, and L. McCluskey. An assessment of features related to phishing websites using an automated technique. *2012 International Conference for Internet Technology and Secured Transactions (ICITST)*, pp. 492–497, 2012. [IEEE Xplore](#).
- [3] S. Shabudin, N. S. Sani, K. A. Z. Ariffin, and M. Aliff. “Feature Selection for Phishing Website Classification,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 4, 2020. DOI: [10.14569/IJACSA.2020.0110477](https://doi.org/10.14569/IJACSA.2020.0110477).